```
#Program
with open("analytics.txt","w+") as f_summary:
    while True:
        user_input=input("Enter 1 to see employment data across industries.\n
        if user_input=="1":
            emp_each_industry()
        elif user_input=="2":
            industry_age_pref()
        elif user_input=="3":
            trend_over_age()
        elif user_input=="4":
            trend_over_occupations()
        elif user_input=="q":
            print("Exiting program.")
            with open("analytics.txt","r") as f_summary:
                for line in f_summary:
                    print(line,end="")
            break #file contents cleared
        else:
            print("\nEnter 1, 2, 3, 4 or'q'\n")
```

Functions

Each codes for the output of a table. Especially useful to make code more concise and organised.

if, elif and else:

Each input will lead to different subsequent programs being run

Analysis Question

Overall Program

Which industry sectors provide the greatest employment opportunities? Are there preferences for age ranges among industries?

while True:

Allows users to loop this program continuously until they enter q which will terminate the loop.

```
Enter'q' to quit program.
1
Enter a year: 2022
Data has been output into out.csv file.
The industry that provides the greatest employment opportunities in 2022 is Services:Wholesale &
Retail Trade.

Enter 1 to see employment data across industries.
Enter 2 to see trends across age groups for an industry over the years.
Enter 3 to see trends across age groups for an occupation over the years.
Enter 4 to see trends across occupations for an age group over the years.
Enter'q' to quit program.
q
Exiting program.
In 2022, the construction hired the most employees from the age range 55 - 59 Years.
The industry that provides the greatest employment opportunities in 2022 is Services:Wholesale &
Retail Trade.
```

Table 1

```python
import csv

def emp_each_industry():
    interested_year=int(input("Enter a year: ")) #convert to integer
    with open("out.csv","w",newline="") as f_out: #specify newline as none
        csv_pointer_outw=csv.writer(f_out)
        header=[[f"Chosen year is {interested_year}"],["Industry","Number of Employees Hired by Each Industry","Percentage of Employees in Each Industry/%"]]
        csv_pointer_outw.writerows(header)
        row=["age range","industry",2024,2023,2022,2021,2020,2019,2018,2017,2016,2015,2014,2013,2012,2011,2010,2009,2008,2007]
        year_index=row.index(interested_year) #refers to "row" list and gives index in row where the targetted year is
        #column 1 in out.csv
        industries=[["Manufacturing"],["Construction"],["Services:Wholesale & Retail Trade"],["Services:Transportation & Storage"],["Services:Accommodation & Food Services"],["Services:Information & Communications"],["Serv
        #column 2 in out.csv
        num_of_employees_hired=[[0] for _ in range(len(industries))] #len(outer_list)=num of sublists
        #column 3 in out.csv
        per_of_emp_each_industry=[[0] for _ in range(len(industries))]
        total_employees=0
        for sublist in industries: #iterations in sublist of nested list
            for industry in sublist: #goes manufacturing, then construction and so on and so forth
                with open("C:/Users/woonc/OneDrive/Desktop/NTU/Decision making with programming and analytics/Python wk 8 indiv/A. Employed Residents Aged 15 Years And Over By Industry And Age Group.csv", "r") as A_in:
                    #open file in each loop iteration so pointer can go back to start of the file
                    csv_pointerA_r=csv.reader(A_in)
                    for line in csv_pointerA_r:
                        if line[1]==industry: #e.g. equals manufacturing
                            a=industries.index(sublist) #so that sublists in three columns are corresponding
                            try:
                                num_of_employees_hired[a][0]+=float(line[year_index]) #num of employees in desired year
                                total_employees+=float(line[year_index])
                            except:
                                pass #when na or - data
        for i,sublist1 in enumerate(num_of_employees_hired):
            for value in sublist1:
                per_of_emp_each_industry[i][0]=(value/total_employees)*100
        for b in range(len(industries)):
            data=[industries[b][0],num_of_employees_hired[b][0],per_of_emp_each_industry[b][0]] #joins three nested list to form one row in each iteration
            #print(data)
            csv_pointer_outw.writerow(data)
    for x,sublist2 in enumerate(per_of_emp_each_industry):
        for value in sublist2:
            if value==max(max(sublist) for sublist in per_of_emp_each_industry): #maximum percentage in the column
                max_industry_index=x #finds index of sublist (industry) with the highest percentage of employees hired
    industry_with_max=industries[max_industry_index][0]
    print("Data has been output into out.csv file.")
    print(f"The industry that provides the greatest employment opportunities in {interested_year} is {industry_with_max}.\n")
    with open("analytics.txt","a") as f_analytics: # use "a" to append, so file contents will not be cleared when opened again
        print(f"The industry that provides the greatest employment opportunities in {interested_year} is {industry_with_max}.",file=f_analytics)
```

input year and code will find index in row list with the year input. To know which index to get employment numbers from the read line

add the year's employment data to the matched sublist

- Three nested lists, each representing a column
  - Each sublist represents a row in the column
- **try** and **except** blocks was used because some occupations have “na” or “-” employees in an age group
- open(“out.csv”, ”w”, **newline= “”**) specify **newline= “”** because open is a pointer that will default add one free new line, but we just need the new line added by the other csv pointer csv.writer()

Table 1

```python
import csv

def emp_each_industry():
    interested_year=int(input("Enter a year: ")) #convert to integer
    with open("out.csv","w",newline="") as f_out: #specify newline as none
        csv_pointer_outw=csv.writer(f_out)
        header=[[f"Chosen year is {interested_year}"],["Industry","Number of Employees Hired by Each Industry","Percentage of Employees in Each Industry/%"]]
        csv_pointer_outw.writerows(header)
        row=["age range","industry",2024,2023,2022,2021,2020,2019,2018,2017,2016,2015,2014,2013,2012,2011,2010,2009,2008,2007]
        year_index=row.index(interested_year) #refers to "row" list and gives index in row where the targetted year is
        #column 1 in out.csv
        industries=[["Manufacturing"],["Construction"],["Services:Wholesale & Retail Trade"],["Services:Transportation & Storage"],["Services:Accommodation & Food Services"],["Services:Information & Communications"],["Serv
        #column 2 in out.csv
        num_of_employees_hired=[[0] for _ in range(len(industries))] #len(outer_list)=num of sublists
        #column 3 in out.csv
        per_of_emp_each_industry=[[0] for _ in range(len(industries))]
        total_employees=0
        for sublist in industries: #iterations in sublist of nested list
            for industry in sublist: #goes manufacturing, then construction and so on and so forth
                with open("C:/Users/woonc/OneDrive/Desktop/NTU/Decision making with programming and analytics/Python wk 8 indiv/A. Employed Residents Aged 15 Years And Over By Industry And Age Group.csv", "r") as A_in:
                    #open file in each loop iteration so pointer can go back to start of the file
                    csv_pointerA_r=csv.reader(A_in)
                    for line in csv_pointerA_r:
                        if line[1]==industry: #e.g. equals manufacturing
                            a=industries.index(sublist) #so that sublists in three columns are corresponding
                            try:
                                num_of_employees_hired[a][0]+=float(line[year_index]) #num of employees in desired year
                                total_employees+=float(line[year_index])
                            except:
                                pass #when na or - data
        for i,sublist1 in enumerate(num_of_employees_hired):
            for value in sublist1:
                per_of_emp_each_industry[i][0]=(value/total_employees)*100
        for b in range(len(industries)):
            data=[industries[b][0],num_of_employees_hired[b][0],per_of_emp_each_industry[b][0]] #joins three nested list to form one row in each iteration
            #print(data)
            csv_pointer_outw.writerow(data)
    for x,sublist2 in enumerate(per_of_emp_each_industry):
        for value in sublist2:
            if value==max(max(sublist) for sublist in per_of_emp_each_industry): #maximum percentage in the column
                max_industry_index=x #finds index of sublist (industry) with the highest percentage of employees hired
    industry_with_max=industries[max_industry_index][0]
    print("Data has been output into out.csv file.")
    print(f"The industry that provides the greatest employment opportunities in {interested_year} is {industry_with_max}.\n")
    with open("analytics.txt","a") as f_analytics: # use "a" to append, so file contents will not be cleared when opened again
        print(f"The industry that provides the greatest employment opportunities in {interested_year} is {industry_with_max}.",file=f_analytics)
```

input year and code will find index in row list with the year input. To know which index to get employment numbers from the read line

add the year's employment data to the matched sublist

Industry is sublist at index x in **“industries”** list

```
Enter 1 to see employment data across industries.
Enter 2 to see trends across age groups for an industry over the years.
Enter 3 to see trends across age groups for an occupation over the years.
Enter 4 to see trends across occupations for an age group over the years.
Enter'q' to quit program.
1
Enter a year: 2024
Data has been output into out.csv file.
The industry that provides the greatest employment opportunities in 2024 is Services:Wholesale & Retail Trade.
```

Table 2

```python
def industry_age_pref():
    input_industry=input("Enter an industry:")
    interested_industry=input_industry.replace(" ","").lower()
    with open("out.csv","w",newline="") as f_out:
        csv_pointer_outw=csv.writer(f_out)
        years=[2024,2023,2022,2021,2020,2019,2018,2017,2016,2015,2014,2013,2012,2011,2010,2009,2008,2007] #to be added in header row
        header=[[f"Chosen industry is {input_industry}",f"Percentage of {input_industry} Employees in this Age Group/%"],["Age Group"]+years]
        csv_pointer_outw.writerows(header)
        # column 1 in out.csv - age_groups
        age_groups=["15 - 19 Years","20 - 24 Years","25 - 29 Years","30 - 34 Years","35 - 39 Years","40 - 44 Years","45 - 49 Years","50 - 54 Years","55 - 59 Years","6
        emp_each_age_group=[[] for _ in range(len(age_groups))]
        # column 2 in out.csv - emp_perc_age_group
        emp_perc_age_group=[[0]*len(years) for _ in range(len(age_groups))]
        occupation_total_over_years=[0]*len(years) # to be used for percentage calculation
        with open("C:/Users/woonc/OneDrive/Desktop/NTU/Decision making with programming and analytics/Python wk 8 indiv/A. Employed Residents Aged 15 Years And Over B
            csv_pointerA_r=csv.reader(A_in)
            for line in csv_pointerA_r: #pointer reads the file
                row_industry=line[1].replace(" ","").lower()
                if row_industry==interested_industry: #e.g. equals manufacturing
                    a=age_groups.index(line[0]) #so that row in age_groups column can match corresponding row in emp_each_age_group
                    emp_each_age_group[a]=line[2:20] #num of employees over the years
                    for i,value in enumerate(line[2:20]):
                        try:
                            occupation_total_over_years[i]+=float(value)
                        except:
                            pass #when na or - data
            for emp_specific_age_grp,sublist1 in enumerate(emp_each_age_group):
                for emp_year,emp_in_year in enumerate(sublist1):
                    try:
                        emp_perc_age_group[emp_specific_age_grp][emp_year]=(float(emp_in_year)/occupation_total_over_years[emp_year])*100 #float()
                    except:
                        pass # when emp_in_year=="na"/"-"
            for age_grp, sublist2 in enumerate(emp_perc_age_group):
                sublist2.insert(0,age_groups[age_grp])
                csv_pointer_outw.writerow(sublist2)
    print("Data has been output into out.csv file.\n")
    year_findmax=int(input(f"Enter a year to find the age range with most hired by {input_industry}:"))
    year_findmax_index=years.index(year_findmax)
    for x,sublist3 in enumerate(emp_perc_age_group):
        for value in sublist3:
            if value==max(sublist[year_findmax_index+1] for sublist in emp_perc_age_group): # finds maximum percentage in 2024 among age groups
                age_range_with_max_index=x
    age_range_with_max=age_groups[age_range_with_max_index] #index 0 looks at year selected
    print(f"In {year_findmax}, the {input_industry} hired the most employees from the age range {age_range_with_max}.\n")
    with open("analytics.txt","a") as f_analytics:
        print(f"In {year_findmax}, the {input_industry} hired the most employees from the age range {age_range_with_max}.",file=f_analytics)
```

.replace(“ ”, “”).lower() on both row_industry and interested_industry before they are compared

Two lists representing two columns--Age Group and Percentage of Employment in the Age Group

- Age group inserted to each sublist in emp_perc_age_group before sublist is written into out.csv

Table 2

```
def industry_age_pref():
    input_industry=input("Enter an industry:")
    interested_industry=input_industry.replace(" ","").lower()
    with open("out.csv","w",newline="") as f_out:
        csv_pointer_outw=csv.writer(f_out)
        years=[2024,2023,2022,2021,2020,2019,2018,2017,2016,2015,2014,2013,2012,2011,2010,2009,2008,2007] #to be added in header row
        header=[[f"Chosen industry is {input_industry}",f"Percentage of {input_industry} Employees in this Age Group/%"],["Age Group"]+years]
        csv_pointer_outw.writerows(header)
        # column 1 in out.csv - age_groups
        age_groups=["15 - 19 Years","20 - 24 Years","25 - 29 Years","30 - 34 Years","35 - 39 Years","40 - 44 Years","45 - 49 Years","50 - 54 Years","55 - 59 Years","6
        emp_each_age_group=[[] for _ in range(len(age_groups))]
        # column 2 in out.csv - emp_perc_age_group
        emp_perc_age_group=[[0]*len(years) for _ in range(len(age_groups))]
        occupation_total_over_years=[0]*len(years) # to be used for percentage calculation
        with open("C:/Users/woonc/OneDrive/Desktop/NTU/Decision making with programming and analytics/Python wk 8 indiv/A. Employed Residents Aged 15 Years And Over B
            csv_pointerA_r=csv.reader(A_in)
            for line in csv_pointerA_r: #pointer reads the file
                row_industry=line[1].replace(" ","").lower()
                if row_industry==interested_industry: #e.g. equals manufacturing
                    a=age_groups.index(line[0]) #so that row in age_groups column
                    emp_each_age_group[a]=line[2:20] #num of employees over the y
                    for i,value in enumerate(line[2:20]):
                        try:
                            occupation_total_over_years[i]+=float(value)
                        except:
                            pass #when na or - data
            for emp_specific_age_grp,sublist1 in enumerate(emp_each_age_group):
                for emp_year,emp_in_year in enumerate(sublist1):
                    try:
                        emp_perc_age_group[emp_specific_age_grp][emp_year]=(float(
                    except:
                        pass # when emp_in_year=="na"/"-"
            for age_grp, sublist2 in enumerate(emp_perc_age_group):
                sublist2.insert(0,age_groups[age_grp])
                csv_pointer_outw.writerow(sublist2)
    print("Data has been output into out.csv file.\n")
    year_findmax=int(input(f"Enter a year to find the age range with most hired by {input_industry}:"))
    year_findmax_index=years.index(year_findmax)
    for x,sublist3 in enumerate(emp_perc_age_group):
        for value in sublist3:
            if value==max(sublist[year_findmax_index+1] for sublist in emp_perc_age_group): # finds maximum percentage in 2024 among age groups
                age_range_with_max_index=x
    age_range_with_max=age_groups[age_range_with_max_index] #index 0 looks at year selected
    print(f"In {year_findmax}, the {input_industry} hired the most employees from the age range {age_range_with_max}.\n")
    with open("analytics.txt","a") as f_analytics:
        print(f"In {year_findmax}, the {input_industry} hired the most employees from the age range {age_range_with_max}.",file=f_analytics)
```

```
Enter 1 to see employment data across industries.
Enter 2 to see trends across age groups for an industry over the
years.
Enter 3 to see trends across age groups for an occupation over the
years.
Enter 4 to see trends across occupations for an age group over the
years.
Enter'q' to quit program.
2
Enter an industry:manufacturing
Data has been output into out.csv file.

Enter a year to find the age range with most hired by manufacturing:
2024
In 2024, the manufacturing hired the most employees from the age range
50 - 54 Years.
```

1

Which industry sectors provide the greatest employment opportunities?

- Wholesale & Retail Trade
- Public Administration & Education

Factors include: government support and funding, Earning potential, Expertise needed

2

Are there preferences for age ranges among industries?

Chosen industry is manufacturing	Percentage of manufacturing Employees in this Age Group/%					
Age Group	**2024**	**2023**	**2022**	**2021**	**2020**	**2019**
15 - 19 Years	0.377003	0.377358	0.489106	0.377003	0.563645	0.373308
20 - 24 Years	1.790763	1.933962	2.356603	2.497644	2.536402	2.286514
25 - 29 Years	5.796418	6.415094	5.558026	6.738926	7.797088	6.392907
30 - 34 Years	7.681433	8.773585	8.403735	9.519321	8.219822	8.679421
35 - 39 Years	9.94345	10.42453	10.76034	11.26296	11.22593	11.71255
40 - 44 Years	13.05372	12.68868	13.73944	14.18473	13.76233	13.71909
45 - 49 Years	12.95947	14.29245	15.51801	14.09048	14.41992	15.63229
50 - 54 Years	15.08011	13.58491	14.05069	11.87559	14.23203	14.83901
55 - 59 Years	12.06409	13.34906	12.0498	11.63996	11.83654	12.0392
60 - 64 Years	9.472196	9.009434	9.337483	9.283695	8.219822	8.119459
65 Years & Over	11.78134	9.150943	7.736772	8.529689	7.186473	6.206253

Table 2

Certain industries hire more of certain age groups, but for some industries, employement is well distributed across age groups. Factors could be that experience is needed in this industry, and also physical ability that can differ with different age groups.

	A	B	C
1	Chosen year is 2024		
2	Industry	Number of Employees Hired by Each Industry	Percentage of Employees in Each Industry/%
3	Manufacturing	212.2	8.970240108
4	Construction	102.9	4.349847819
5	Services:Wholesale & Retail Trade	335.9	14.19935746
6	Services:Transportation & Storage	198	8.369969564
7	Services:Accommodation & Food Services	147.5	6.235204599
8	Services:Information & Communications	139.2	5.884342239
9	Services:Financial & Insurance Services	239.3	10.11582685
10	Services:Real Estate Services	56.2	2.375718634
11	Services:Professional Services	197.2	8.336151505
12	Services:Administrative & Support Services	124	5.241799121
13	Services:Public Administration & Education	305.6	12.91849848
14	Services:Health & Social Services	174.5	7.376564085
15	Services:Arts, Entertainment & Recreation	37.9	1.602130538
16	Services:Other Community, Social & Personal Services	73	3.085897869
17	Others	22.2	0.938451133

Table 1

```
Enter 1 to see employment data across industries.
Enter 2 to see trends across age groups for an industry over the years.
Enter 3 to see trends across age groups for an occupation over the years.
Enter 4 to see trends across occupations for an age group over the years.
Enter'q' to quit program.
q
Exiting program.
The industry that provides the greatest employment opportunities in 2024 is Services:Wholesale &
Retail Trade.

In 2024, the manufacturing hired the most employees from the age range 50 - 54 Years.
```

Answering Analysis Question

```python
def trend_over_age():
    input_occupation=input("Enter the occupation:")
    interested_occupation=input_occupation.replace(" ","").lower() #to remove spaces and standardise format for comparison
    with open("out.csv","w",newline="") as f_out:
        csv_pointer_outw=csv.writer(f_out)
        years=[2024,2023,2022,2021,2020,2019,2018,2017,2016,2015,2014,2013,2012,2011,2010,2009,2008,2007]
        header=[[f"Chosen Occupation is {input_occupation}",f"Percentage of {input_occupation} Employees in this Age Group/%"],["Age Group"]+years]
        csv_pointer_outw.writerows(header)
        age_groups=["15 - 19 Years","20 - 24 Years","25 - 29 Years","30 - 34 Years","35 - 39 Years","40 - 44 Years","45 - 49 Years","50 - 54 Years","55
        emp_each_age_group=[[] for _ in range(len(age_groups))]
        emp_perc_age_group=[[0]*len(years) for _ in range(len(age_groups))]
        occupation_total_over_years=[0]*len(years)
        with open("C:/Users/woonc/OneDrive/Desktop/NTU/Decision making with programming and analytics/Python wk 8 indiv/B. Employed Residents Aged 15 Ye
            csv_pointerB_r=csv.reader(B_in)
            for line in csv_pointerB_r: #pointer reads the file
                row_occupation=line[1].replace(" ","").lower() #e.g. "associateprofessionals&technicians"
                if row_occupation==interested_occupation:
                    a=age_groups.index(line[0])
                    emp_each_age_group[a]=line[2:20] #num of employees over the years
                    for i,value in enumerate(line[2:20]):
                        try:
                            occupation_total_over_years[i]+=float(value)
                        except:
                            pass
            #print(emp_each_age_group)
            for emp_specific_age_grp,sublist2 in enumerate(emp_each_age_group):
                for emp_year,emp_in_year in enumerate(sublist2):
                    try:
                        if emp_in_year=="na":
                            emp_perc_age_group[emp_specific_age_grp][emp_year]="na"
                        else:
                            emp_perc_age_group[emp_specific_age_grp][emp_year]+=(float(emp_in_year)/occupation_total_over_y
                    except: #some age groups have zero employees in certain years in the chosen occupation
                        emp_perc_age_group[emp_specific_age_grp][emp_year]=0
            for age_grp, sublist3 in enumerate(emp_perc_age_group):
                sublist3.insert(0,age_groups[age_grp])
                csv_pointer_outw.writerow(sublist3)
    print("Data has been output into out.csv file\n")
```

Table 3

“Users may pick an occupation and be presented with the trends over the years by age groups or pick an age group and show the trend over the years by occupations.”

```python
def trend_over_occupations():
    input_age_grp=input("Enter an age group: ")
    interested_age_grp=input_age_grp.replace(" ","").lower() #replace and lower to standardise
    with open("out.csv","w",newline="") as f_out: #newline change from default
        csv_pointer_outw=csv.writer(f_out)
        years=[2024,2023,2022,2021,2020,2019,2018,2017,2016,2015,2014,2013,2012,2011,2010,2009,2008,2007]
        header=[[f"Chosen Age Group is {input_age_grp}",f"Percentage of Employees Aged {interested_age_grp} in this Occupation/%"],["Occupation"]+years]
        csv_pointer_outw.writerows(header)
        occupations=["Associate Professionals & Technicians","Cleaners, Labourers & Related Workers","Clerical Support Workers","Craftsmen & Related Tra
        emp_each_occupation=[[] for _ in range(len(occupations))] #len(outer_list)=num of sublists
        emp_perc_occupation=[[0]*len(years) for _ in range(len(occupations))]
        age_grp_total_over_years=[0]*len(years)
        with open("C:/Users/woonc/OneDrive/Desktop/NTU/Decision making with programming and analytics/Python wk 8 indiv/B. Employed Residents Aged 15 Ye
            csv_pointerB_r=csv.reader(B_in)
            for line in csv_pointerB_r: #pointer reads the file
                row_age_grp=line[0].replace(" ","").lower()
                if row_age_grp==interested_age_grp:
                    a=occupations.index(line[1].strip())
                    emp_each_occupation[a]=line[2:20] #num of employees over the years
                    for i,value in enumerate(line[2:20]):
                        try:
                            age_grp_total_over_years[i]+=float(value)
                        except:
                            pass
            for emp_specific_occupation,sublist2 in enumerate(emp_each_occupation):
                for emp_year,emp_in_year in enumerate(sublist2):
                    try:
                        if emp_in_year=="na":
                            emp_perc_occupation[emp_specific_occupation][emp_year]="na"
                        else:
                            emp_perc_occupation[emp_specific_occupation][emp_year]+=(float(emp_in_year)/age_grp_total_over_years[emp_year])*100 #float()
                    except: #some age groups have zero employees in certain years in the chosen occupation
                        emp_perc_occupation[emp_specific_occupation][emp_year]=0
            for occupation, sublist3 in enumerate(emp_perc_occupation):
                sublist3.insert(0,occupations[occupation])
                csv_pointer_outw.writerow(sublist3)
    print("Data has been output into out.csv file.\n")
```

Table 4

Table 3

Chosen Occupation is Managers & Administrators (Including Working Proprietors)	Percentage of Managers & Administrators (Including Working Proprietors) Employees in this Age Group/%								
Age Group	2024	2023	2022	2021	2020	2019	2018	2017	2016
15 - 19 Years	na	na	na	na	na	na	na	na	na
20 - 24 Years	0.148221	0.162294	0.26974	0.291314	0.135759	0.246643	0.204858	0.286615	0.266746
25 - 29 Years	2.050395	3.218826	2.942619	3.257415	2.579419	2.384215	2.69242	2.46489	2.786011
30 - 34 Years	8.522727	9.521233	10.07847	10.30191	8.932935	8.769526	8.66257	8.827744	10.37344
35 - 39 Years	15.66206	14.71463	13.95292	14.59216	16.01955	15.62072	15.33509	14.96131	15.32306
40 - 44 Years	17.58893	16.98675	17.87641	17.47881	18.11024	17.95012	19.54931	19.71912	19.26497
45 - 49 Years	17.63834	17.47363	17.90093	17.13453	18.97909	19.07372	18.55429	18.544	17.81269
50 - 54 Years	15.51383	15.71544	14.39431	13.05614	13.92886	14.14086	14.42786	13.92949	13.78186
55 - 59 Years	10.15316	10.2786	10.88769	9.692797	10.39913	10.41381	10.03805	10.31814	10.13634
60 - 64 Years	6.571146	6.410603	6.130456	7.627119	5.837632	6.577144	6.175007	6.362855	6.046236
65 Years & Over	6.151186	5.517988	5.566454	6.567797	5.077383	4.823239	4.36055	4.585841	4.208654

Table 4

Chosen Age Group is 15-19 years	Percentage of Employees Aged 15-19years in this Occupation/%					
Occupation	2024	2023	2022	2021	2020	2019
Associate Professionals & Technicians	8.76494	9.966777	10.8209	8.148148	6.439394	4.892966
Cleaners, Labourers & Related Workers	7.171315	6.976744	6.343284	10.74074	6.818182	5.810398
Clerical Support Workers	12.3506	11.62791	12.68657	20.74074	13.25758	16.20795
Craftsmen & Related Trade Workers	2.390438	0.664452	1.865672	0.37037	1.515152	0.30581
Managers & Administrators (Including Working Proprietors)	na	na	na	na	na	na
Plant & Machine Operators & Assemblers	1.992032	0.664452	4.104478	1.851852	3.030303	2.140673
Professionals	2.788845	1.66113	0.746269	0.740741	0.378788	0.917431
Service & Sales Workers	25.89641	33.88704	25.37313	24.44444	23.10606	26.6055
Others	38.64542	34.5515	38.0597	32.96296	45.45455	43.11927

Thank You